

Part 1

1.1

relation a

Employee(EmpID, SSN, Email, Phone, Name, Department, Salary)

1. List at least 6 different superkeys

1. {empID}
2. {SSN}
3. {Email}
4. {empID , name}
5. {SNN , departament}
6. {Email , departament}

2. Identify all candidate keys

{email}

{empID}

{snn}

3. Which candidate key would you choose as primary key and why?

{empID} because it never changes

4. Can two employees have the same phone number? Justify your answer based on the data shown

no, according to the table, every employe has their own phone which dosen,t duplicate

relation b

Registration(StudentID, CourseCode, Section, Semester, Year, Grade, Credits)

1. Determine the minimum attributes needed for the primary key

{studentID, CoureCode, semester, year}

2. Explain why each attribute in your primary key is necessary

studentID - every student as their own Id and determine which is this student

CoureCode - studet may pass some courses

semester, year - student can pass the same course in ane semester,year

3. Identify any additional candidate keys (if they exist)
{studentID, Section, Year, Semestr}

1.2

Given Tables:

Student(StudentID, Name, Email, Major, AdvisorID)

Professor(ProfID, Name, Department, Salary)

Course(CourseID, Title, Credits, DepartmentCode)

Department(DeptCode, DeptName, Budget, ChairID)

Enrollment(StudentID, CourseID, Semester, Grade)

1. Identify all foreign key relationships

1. student -> Professor

studentAdvisorID, ProfID

2. student -> course

StudentID, CourseID

3. Professor -> department

ProfID, ChairID

4. course -> department

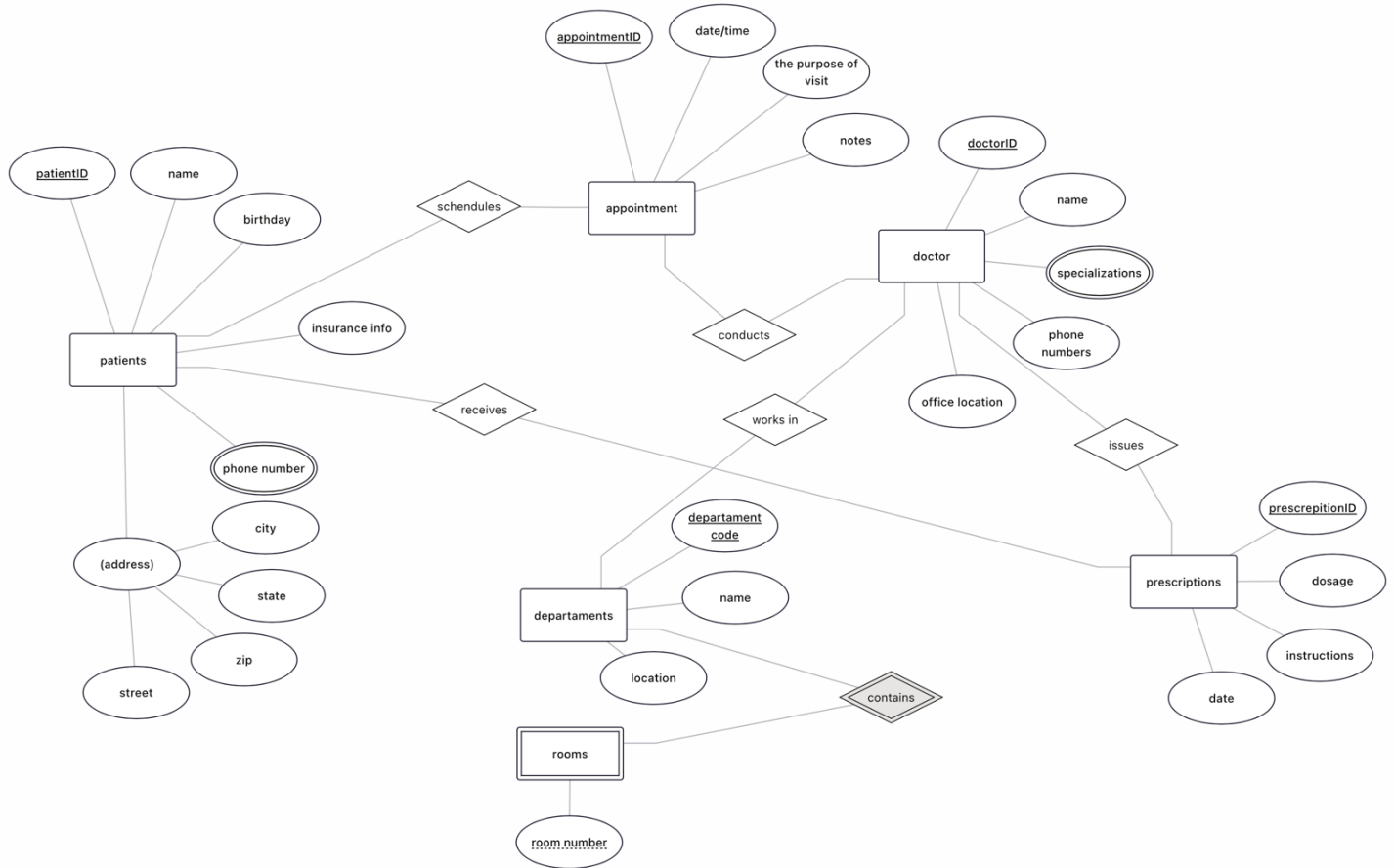
departmentCode, deptCode

5. Professor -> course

profID, courseID

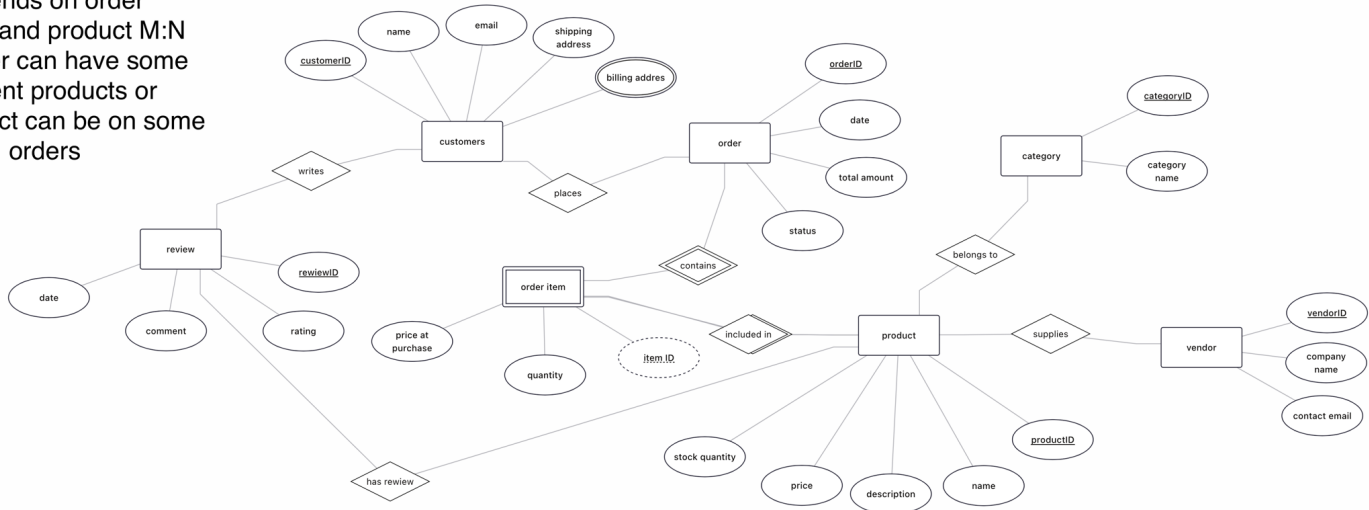
part2

2.1



2.2

- 1:
2: order item because it depends on order
3: order and product M:N
one order can have some different products or
one product can be on some orders



Part4

4.1

StudentProject(StudentID, StudentName, StudentMajor, ProjectID, ProjectTitle, ProjectType, SupervisorID, SupervisorName, SupervisorDept, Role, HoursWorked, StartDate, EndDate)

Your Tasks: 1. Identify functional dependencies: List all FDs in the format $A \rightarrow B$

2. Identify problems: - What redundancy exists in this table? - Give specific examples of update, insert, and delete anomalies

3. Apply 1NF: Are there any 1NF violations? How would you fix them?

4. Apply 2NF: - What is the primary key of this table?

- Identify any partial dependencies - Show the 2NF decomposition

5. Apply 3NF: - Identify any transitive dependencies

- Show the final 3NF decomposition with all table schemas

1. StudentID $\rightarrow$ StudentName, StudentMajor

ProjectID $\rightarrow$ ProjectTitle, ProjectType

SupervisorID $\rightarrow$ SupervisorName, SupervisorDept

StudentID, ProjectID $\rightarrow$ Role, HoursWorked, StartDate, EndDate

2. redundancy - name and speciality duplicate for each new project.

update- change name of supervisor needs update in all strings

insert- we cant add student without a project or project without student because (studentID and ProjectID) are primary key

delete- if we delete students data we may delete information about project too

3. status , action

4. primary key - studentID, ProjectID

partial dependences:

studentID $\rightarrow$ StudentName, StudentMajor

ProjectID $\rightarrow$ ProjectTitle, ProjectType, SupervisorID, SupervisorDept, SupervisorName

2N: student(studentID, StudentName, StudentMajor)

project(ProjectID, ProjectTitle , ProjectType, SupervisorID, SupervisorName, StudentProject)

studentproject(studentID, ProjectID, Role, HoursWorked, StartDate, EndDate)

5. transitive dependences: SupervisorID $\rightarrow$ SupervisorName, SupervisorDept

3N: Student(StudentID, StudentName, StudentMajor) —StudentID

Supervisor(SupervisorID, SupervisorName, SupervisorDept) —SupervisorID

Project(ProjectID, ProjectTitle, ProjectType, SupervisorID) —ProjectID,SupervisorID

StudentProjectAssignment(StudentID, ProjectID, Role, HoursWorked, StartDate, EndDate)—(StudentID, ProjectID)

4.2

Given Table:

CourseSchedule(StudentID, StudentMajor, CourseID, CourseName,
InstructorID, InstructorName, TimeSlot, Room, Building)

Business Rules:

- Each student has exactly one major
- Each course has a fixed name
- Each instructor has exactly one name
- Each time slot in a room determines the building (rooms are unique across campus)
- Each course section is taught by one instructor at one time in one room
- A student can be enrolled in multiple course sections

Your Tasks: 1. Determine the primary key of this table (hint: this is tricky!) 2. List all functional dependencies 3. Check if the table is in BCNF 4. If not in BCNF, decompose it to BCNF showing your work 5. Explain any potential loss of information in your decomposition

1. studentID, courseID

2. studentID -> StudentMajor

courseID -> CourseName, InstructorID, TimeSlot, Room

InstructorID -> InstructorName

room -> building

3. no

4. Student(StudentID, StudentMajor) StudentID

Instructor(InstructorID, InstructorName) InstructorID

RoomLocation(Room, Building) Room

Course(CourseID, CourseName, InstructorID, TimeSlot, Room) CourseID

InstructorID -> Instructor(InstructorID)

Room -> RoomLocation(Room)

StudentEnrollment(StudentID, CourseID) (StudentID, CourseID)

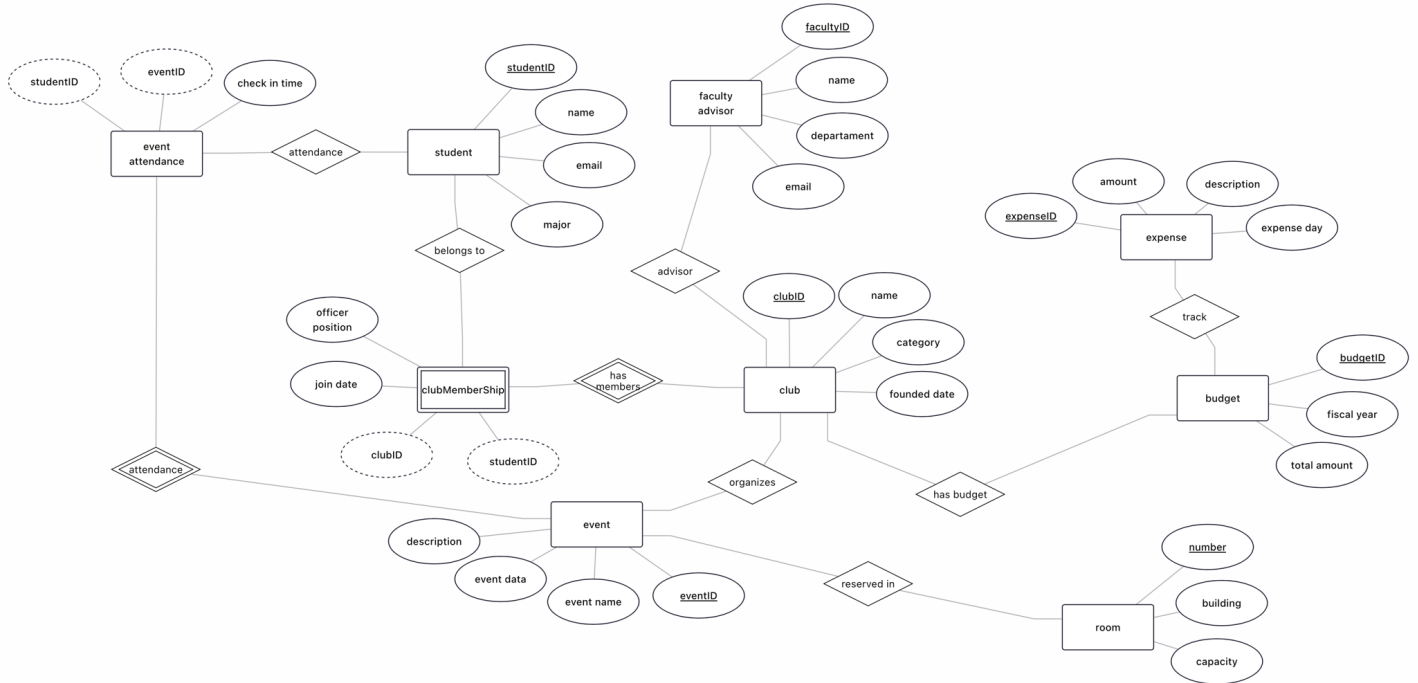
StudentID -> Student(StudentID)

CourseID -> Course(CourseID)

5. join with primary key we can restore the original table without adding false strings
dependency preservation all information saves inside new tables

part5

1.



2. student(studentID, name, email, major)

faculty advisor(facultyID, name, department, email)

club(clubID, name, category, foundedDate)

clubMembership(studentID, clubID, joinDate, officerPosition)

studentID -> student(studentID)

clubID -> club(clubID)

room(number, building, capacity)

event(eventID, eventName, eventData, description, clubID, roomID)

clubID -> club(clubID)

roomID -> room(roomID)

eventAttendance(studentID, eventID, checkInTime)

eventID -> event(eventID)

studentID -> student(studentID)

budget(budgetID, fiscalYear, totalAmount, clubID)

clubID -> club(clubID)

expense(expenseID, amount, description, expenseDay, budgetID)

budgetID -> budget(budgetID)

3. design decision: officerPosition inside the clubMembership

alternative: create a separate officerPosition entity and dedicated ClubOfficers here

it will be take more difficulty

4. list all students majoring in Information Systems (IS) who attended a specific club event

find the club event that is associated with the largest overall club budget

find all events scheduled to take place in a specific room